

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное
учреждение
высшего образования
«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

Институт Перспективной инженерии
Департамент цифровых и робототехнических систем и электроники

**ОТЧЕТ
ПО ЛАБОРАТОРНОЙ РАБОТЕ № 1
дисциплины «ООП»**

Выполнил: Мендеш Пашкоал Педру
3 курс, группа ИВТ-б-о-24-1,
09.03.01 «Информатика и Вычислительная
техника», направленность (профиль)
«Программное обеспечение средств
вычислительной техники и
автоматизированных систем», очная форма
обучения

(подпись)

Руководитель практики:
Воронкин Роман А. доцент факультета
цифровых, робототехнических систем и
электроники института перспективной
инженерии.

(подпись)

Отчет защищен с оценкой _____ Дата защиты _____

Ставрополь, 2026 г.

Тема: Элементы объектно ориентированного программирования в языке Python

Цель работы: приобретение навыков по работе с классами и объектами при написании программ с помощью языка программирования Python версии 3.x.

Методика выполнения работы

1. Изучение теоретического материала
2. Создание репозитория на GitHub
3. Клонирование

репозитория:

<https://github.com/Pascoalpm/python-oop-labs.git>

Comentado [PM1]: <https://github.com/Pascoalpm/python-oop-labs.git>

4. Настройка файла .gitignore
5. Изучение примеров лабораторной работы
6. Выполнение индивидуальных заданий

Примеры

Пример 1: Класс Book (атрибуты класса)

```
class Book:
    material = "paper"
    cover = "paperback"
    all_books = []

print("=== Пример 1: Класс Book ===\n")
print(Book.material)
print(Book.cover)
print(Book.all_books)

my_book = Book()
print(my_book.material)
print(my_book.cover)
print(my_book.all_books)

=== Пример 1: Класс Book ===

paper
paperback
[]
paper
paperback
[]
```

Рисунок 1. Результат примера 1 (Класс Book)

Пример 2: Класс River

```
class River:
    all_rivers = []

    def __init__(self, name, length):
        self.name = name
        self.length = length
        River.all_rivers.append(self)

print("=== Пример 2: Класс River ===\n")
volga = River("Волга", 3530)
seine = River("Сена", 776)
nile = River("Нил", 6852)

for river in River.all_rivers:
    print(river.name)

def get_info(self):
    print("длина {0} равна {1} км".format(self.name, self.length))

River.get_info = get_info

volga.get_info()
seine.get_info()
nile.get_info()
```

=== Пример 2: Класс River ===

Волга
Сена
Нил
длина Волга равна 3530 км
длина Сена равна 776 км
длина Нил равна 6852 км

Рисунок 2. Результат примера 2 (Класс River)

Пример 3: Класс Pet

```
class Pet:
    kind = "mammal"
    n_pets = 0
    pet_names = []

    def __init__(self, spec, name):
        self.spec = spec
        self.name = name
        self.legs = 4

print("=== Пример 3: Класс Pet ===\n")
tom = Pet("cat", "Tom")
avocado = Pet("dog", "Avocado")
ben = Pet("goldfish", "Benjamin")

print("Pet.n_pets =", Pet.n_pets)
print("tom.n_pets =", tom.n_pets)
print("avocado.n_pets =", avocado.n_pets)
print("ben.n_pets =", ben.n_pets)
```

```

Pet.n_pets = 3
print("\nПосле Pet.n_pets = 3:")
print("Pet.n_pets =", Pet.n_pets)
print("tom.n_pets =", tom.n_pets)
print("avocado.n_pets =", avocado.n_pets)
print("ben.n_pets =", ben.n_pets)

print("\nИзменение kind для ben:")
ben.kind = "fish"
print("Pet.kind =", Pet.kind)
print("tom.kind =", tom.kind)
print("avocado.kind =", avocado.kind)
print("ben.kind =", ben.kind)

```

=== Пример 3: Класс Pet ===

```

Pet.n_pets = 0
tom.n_pets = 0
avocado.n_pets = 0
ben.n_pets = 0

После Pet.n_pets = 3:
Pet.n_pets = 3
tom.n_pets = 3
avocado.n_pets = 3
ben.n_pets = 3

Изменение kind для ben:
Pet.kind = mammal
tom.kind = mammal
avocado.kind = mammal
ben.kind = fish

```

Рисунок 3. Результат примера 3 (Класс Pet)

Пример 4: Класс Ship

```

class Ship:
    def __init__(self, name, capacity):
        self.name = name
        self.capacity = capacity
        self.cargo = 0

    def sail(self):
        print("{} has sailed!".format(self.name))

    def load_cargo(self, weight):
        if self.cargo + weight <= self.capacity:
            self.cargo += weight
            print("Loaded {} tons".format(weight))
        else:
            print("Cannot load that much")

    def unload_cargo(self, weight):
        if self.cargo - weight >= 0:
            self.cargo -= weight
            print("Unloaded {} tons".format(weight))
        else:
            print("Cannot unload that much")

    def free_space(self):
        return self.capacity - self.cargo

```

```

print("=== Пример 4: Класс Ship ===\n")
black_pearl = Ship("Black Pearl", 800)

black_pearl.sail()
black_pearl.load_cargo(600)
black_pearl.unload_cargo(400)
black_pearl.load_cargo(700)
black_pearl.unload_cargo(300)

print("\nТекущий груз:", black_pearl.cargo)
print("Свободно места:", black_pearl.free_space())

```

```

=== Пример 4: Класс Ship ===

Black Pearl has sailed!
Loaded 600 tons
Unloaded 400 tons
Cannot load that much
Cannot unload that much

Текущий груз: 200
Свободно места: 600

```

Рисунок 4. Результат примера 4 (Класс Ship)

Пример 5: Класс Rectangle (@property)

```

class Rectangle:
    def __init__(self, width, height):
        self.__width = width
        self.__height = height

    @property
    def width(self):
        return self.__width

    @width.setter
    def width(self, w):
        if w > 0:
            self.__width = w
        else:
            raise ValueError

    @property
    def height(self):
        return self.__height

    @height.setter
    def height(self, h):
        if h > 0:
            self.__height = h
        else:
            raise ValueError

    def area(self):
        return self.__width * self.__height

print("=== Пример 5: Класс Rectangle (@property) ===\n")
rect = Rectangle(10, 20)

print("Чтение через property:")
print("rect.width =", rect.width)

```

```

print("rect.height =", rect.height)
print("rect.area() =", rect.area())

print("\nИзменение через property:")
rect.width = 50
rect.height = 70

print("rect.width =", rect.width)
print("rect.height =", rect.height)
print("rect.area() =", rect.area())

```

=== Пример 5: Класс Rectangle (@property) ===

Чтение через property:

```

rect.width = 10
rect.height = 20
rect.area() = 200

```

Изменение через property:

```

rect.width = 50
rect.height = 70
rect.area() = 3500

```

Рисунок 5. Результат примера 5 (Класс Rectangle)

Пример 6: Класс Rational

```

class Rational:
    def __init__(self, a=0, b=1):
        a = int(a)
        b = int(b)
        if b == 0:
            raise ValueError()
        self.__numerator = abs(a)
        self.__denominator = abs(b)
        self.__reduce()

    def __reduce(self):
        def gcd(a, b):
            if a == 0:
                return b
            elif b == 0:
                return a
            elif a >= b:
                return gcd(a % b, b)
            else:
                return gcd(a, b % a)
        c = gcd(self.__numerator, self.__denominator)
        self.__numerator //= c
        self.__denominator //= c

    @property
    def numerator(self):
        return self.__numerator

    @property
    def denominator(self):
        return self.__denominator

    def display(self):
        print(f"{self.__numerator}/{self.__denominator}")

```

```

def add(self, rhs):
    if isinstance(rhs, Rational):
        a = self.numerator * rhs.denominator + self.denominator *
rhs.numerator
        b = self.denominator * rhs.denominator
        return Rational(a, b)
    else:
        raise ValueError()

print("=== Пример 6: Класс Rational ===\n")
r1 = Rational(3, 4)
print("r1 =", end=" ")
r1.display()

r2 = Rational(5, 6)
print("r2 =", end=" ")
r2.display()

r3 = r1.add(r2)
print("r1 + r2 =", end=" ")
r3.display()

```

```

=== Пример 6: Класс Rational ===

r1 = 3/4
r2 = 5/6
r1 + r2 = 19/12

```

Рисунок 6. Результат примера 6 (Класс Rational)

Индивидуальные задания

Задание 1

Парой называется класс с двумя полями, которые обычно имеют имена `first` и `second`. Требуется реализовать тип данных с помощью такого класса. Во всех заданиях обязательно должны присутствовать:

метод инициализации `__init__` ; метод должен контролировать значения аргументов

- на корректность;
- ввод с клавиатуры `read` ;
- вывод на экран `display` .

Реализовать внешнюю функцию с именем `make_тип()` , где `тип` — тип реализуемой

структуры. Функция должна получать в качестве аргументов значения для полей структуры и возвращать структуру требуемого типа. При передаче ошибочных

параметров следует выводить сообщение и заканчивать работу.

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-

class Power:

    def __init__(self, first: float = 0.0, second: int = 0):

        try:
            self.__first = float(first)
            self.__second = int(second)
        except (ValueError, TypeError):
            print("Ошибка: неверный тип аргументов")
            print("first должно быть числом, second - целым числом")
            exit(1)

    def read(self, prompt: str = None):
        if prompt is None:
            line = input("Введите основание и степень (через пробел): ")
        else:
            line = input(prompt)

        parts = line.strip().split()
        if len(parts) != 2:
            print("Ошибка: необходимо ввести два числа через пробел")
            exit(1)

        try:
            self.__first = float(parts[0])
            self.__second = int(parts[1])
        except ValueError:
            print("Ошибка: неверный формат чисел")
            exit(1)

    def display(self):
        """Вывод значений на экран."""
        print(f"first = {self.__first}, second = {self.__second}")

    def power(self) -> float:

        return self.__first ** self.__second

# Свойства для доступа к полям (дополнительно)
@property
def first(self) -> float:
    return self.__first

@property
def second(self) -> int:
    return self.__second
```



```

def make_power(first: float, second: int) -> Power:

    if not isinstance(first, (int, float)):
        print("Ошибка: first должно быть числом")
        exit(1)
    if not isinstance(second, int):
        print("Ошибка: second должно быть целым числом")
        exit(1)

    return Power(first, second)

if __name__ == "__main__":
    # Демонстрация работы класса

    # 1. Создание через конструктор
    print("=== Демонстрация 1: Конструктор ===")
    p1 = Power(2.5, 3)
    p1.display()
    print(f"2.5 ^ 3 = {p1.power()}")
    print()

    # 2. Создание через внешнюю функцию
    print("=== Демонстрация 2: make_power() ===")
    p2 = make_power(4.0, -2)
    p2.display()
    print(f"4.0 ^ -2 = {p2.power()}")
    print()

    # 3. Ввод с клавиатуры
    print("=== Демонстрация 3: Ввод с клавиатуры ===")
    p3 = Power()
    p3.read("Введите основание и степень: ")
    p3.display()
    print(f"{p3.first} ^ {p3.second} = {p3.power()}")
    print()

    # 4. Граничные случаи
    print("=== Демонстрация 4: Граничные случаи ===")

    # Нулевая степень
    p4 = Power(5.0, 0)
    print(f"5.0 ^ 0 = {p4.power()}")

    # Отрицательное основание с чётной степенью
    p5 = Power(-3.0, 2)
    print(f"(-3.0) ^ 2 = {p5.power()}")

    # Отрицательное основание с нечётной степенью
    p6 = Power(-3.0, 3)
    print(f"(-3.0) ^ 3 = {p6.power()}")

    # Дробное основание
    p7 = Power(0.5, 4)
    print(f"0.5 ^ 4 = {p7.power()}")

    # Отрицательная степень (дробь)
    p8 = Power(2.0, -3)
    print(f"2.0 ^ -3 = {p8.power()}")

```

```

=== Демонстрация 1: Конструктор ===
first = 2.5, second = 3
2.5 ^ 3 = 15.625

=== Демонстрация 2: make_power() ===
first = 4.0, second = -2
4.0 ^ -2 = 0.0625

=== Демонстрация 3: Ввод с клавиатуры ===
Введите основание и степень: 2 4
first = 2.0, second = 4
2.0 ^ 4 = 16.0

=== Демонстрация 4: Граничные случаи ===
5.0 ^ 0 = 1.0
(-3.0) ^ 2 = 9.0
(-3.0) ^ 3 = -27.0
0.5 ^ 4 = 0.0625
2.0 ^ -3 = 0.125

```

Рисунок 7. Результат заданий 1

Задание 2

Составить программу с использованием классов и объектов для решения задачи. Во всех заданиях, помимо указанных в задании операций, обязательно должны быть реализованы следующие методы:

- метод инициализации `__init__` ;
- ввод с клавиатуры `read` ;
- вывод на экран `display` .

1. Создать класс `Vector3D`, задаваемый тройкой координат.

Обязательно должны быть реализованы: сложение и вычитание векторов, скалярное произведение векторов, умножение на скаляр, сравнение векторов, вычисление длины вектора, сравнение длины векторов.

```

import math

class Vector3D:

    def __init__(self, x: float = 0.0, y: float = 0.0, z: float = 0.0):

        try:
            self.__x = float(x)

```

```

        self.__y = float(y)
        self.__z = float(z)
    except (ValueError, TypeError):
        print("Ошибка: все координаты должны быть числами")
        exit(1)

def read(self, prompt: str = None):

    if prompt is None:
        line = input("Введите координаты x y z (через пробел): ")
    else:
        line = input(prompt)

    parts = line.strip().split()
    if len(parts) != 3:
        print("Ошибка: необходимо ввести три числа через пробел")
        exit(1)

    try:
        self.__x = float(parts[0])
        self.__y = float(parts[1])
        self.__z = float(parts[2])
    except ValueError:
        print("Ошибка: неверный формат чисел")
        exit(1)

def display(self, name: str = "Vector"):
    """Вывод вектора на экран."""
    print(f"{name} = ({self.__x}, {self.__y}, {self.__z})")

# === Арифметические операции ===

def add(self, other: 'Vector3D') -> 'Vector3D':

    if not isinstance(other, Vector3D):
        raise ValueError("Операнд должен быть Vector3D")
    return Vector3D(
        self.__x + other.__x,
        self.__y + other.__y,
        self.__z + other.__z
    )

def sub(self, other: 'Vector3D') -> 'Vector3D':

    if not isinstance(other, Vector3D):
        raise ValueError("Операнд должен быть Vector3D")
    return Vector3D(
        self.__x - other.__x,
        self.__y - other.__y,
        self.__z - other.__z
    )

def dot(self, other: 'Vector3D') -> float:
    """
    Скалярное произведение векторов.
    (x1, y1, z1) · (x2, y2, z2) = x1*x2 + y1*y2 + z1*z2
    """
    if not isinstance(other, Vector3D):
        raise ValueError("Операнд должен быть Vector3D")
    return (self.__x * other.__x +
            self.__y * other.__y +

```

```

        self.__z * other.__z)

def scalar_mul(self, scalar: float) -> 'Vector3D':

    try:
        s = float(scalar)
    except (ValueError, TypeError):
        raise ValueError("Скаляр должен быть числом")
    return Vector3D(
        self.__x * s,
        self.__y * s,
        self.__z * s
    )

# === Операции сравнения ===

def equals(self, other: 'Vector3D') -> bool:
    """Сравнение векторов на равенство."""
    if not isinstance(other, Vector3D):
        return False
    return (self.__x == other.__x and
            self.__y == other.__y and
            self.__z == other.__z)

def greater(self, other: 'Vector3D') -> bool:
    return self.length() > other.length()

def less(self, other: 'Vector3D') -> bool:
    return self.length() < other.length()

# === Длина вектора ===

def length(self) -> float:
    """Вычисление длины (модуля) вектора."""
    return math.sqrt(self.__x ** 2 + self.__y ** 2 + self.__z **
2)

def length_squared(self) -> float:
    """Квадрат длины вектора (без извлечения корня)."""
    return self.__x ** 2 + self.__y ** 2 + self.__z ** 2

# === Свойства для доступа к координатам ===
@property
def x(self) -> float:
    return self.__x

@property
def y(self) -> float:
    return self.__y

@property
def z(self) -> float:
    return self.__z

def make_vector3d(x: float, y: float, z: float) -> Vector3D:
    return Vector3D(x, y, z)

```

```

if __name__ == "__main__":
    # Демонстрация работы класса

    print("          РАБОТА С ТРЁХМЕРНЫМИ ВЕКТОРАМИ")

    # 1. Создание векторов через конструктор
    print("\n=== 1. Создание векторов ===")
    v1 = Vector3D(1, 2, 3)
    v2 = Vector3D(4, 5, 6)
    v1.display("v1")
    v2.display("v2")

    # 2. Сложение и вычитание
    print("\n=== 2. Сложение и вычитание ===")
    v_sum = v1.add(v2)
    v_sum.display("v1 + v2")

    v_sub = v1.sub(v2)
    v_sub.display("v1 - v2")

    # 3. Скалярное произведение
    print("\n=== 3. Скалярное произведение ===")
    dot_product = v1.dot(v2)
    print(f"v1 · v2 = {dot_product}")

    # 4. Умножение на скаляр
    print("\n=== 4. Умножение на скаляр ===")
    v_scaled = v1.scalar_mul(2.5)
    v_scaled.display("v1 * 2.5")

    # 5. Длина вектора
    print("\n=== 5. Длина вектора ===")
    print(f"|v1| = {v1.length():.4f}")
    print(f"|v2| = {v2.length():.4f}")

    # 6. Сравнение векторов
    print("\n=== 6. Сравнение векторов ===")
    print(f"v1 == v2: {v1.equals(v2)}")
    print(f"v1 > v2 (по длине): {v1.greater(v2)}")
    print(f"v1 < v2 (по длине): {v1.less(v2)}")

    # 7. Создание через внешнюю функцию
    print("\n=== 7. Создание через make_vector3d() ===")
    v3 = make_vector3d(0, 0, 0)
    v3.display("v3 (нулевой вектор)")
    print(f"Длина нулевого вектора: {v3.length()}")

    # 8. Ввод с клавиатуры
    print("\n=== 8. Ввод с клавиатуры ===")
    v4 = Vector3D()
    v4.read("Введите координаты x y z: ")
    v4.display("v4 (введённый)")

    # 9. Сравнение длины векторов с нулём (проверка)
    print("\n=== 9. Проверка сравнения ===")
    v_zero = Vector3D(0, 0, 0)
    v_test = Vector3D(3, 4, 0)
    print(f"|v_test| = {v_test.length()} (должно быть 5.0)")
    print(f"v_test > zero: {v_test.greater(v_zero)}")
    print(f"v_test == zero: {v_test.equals(v_zero)}")

```

```

РАБОТА С ТРЁХМЕРНЫМИ ВЕКТОРАМИ

=== 1. Создание векторов ===
v1 = (1.0, 2.0, 3.0)
v2 = (4.0, 5.0, 6.0)

=== 2. Сложение и вычитание ===
v1 + v2 = (5.0, 7.0, 9.0)
v1 - v2 = (-3.0, -3.0, -3.0)

=== 3. Скалярное произведение ===
v1 · v2 = 32.0
|
=== 4. Умножение на скаляр ===
v1 * 2.5 = (2.5, 5.0, 7.5)

=== 5. Длина вектора ===
|v1| = 3.7417
|v2| = 8.7750

=== 6. Сравнение векторов ===
v1 == v2: False
v1 > v2 (по длине): False
v1 < v2 (по длине): True

=== 7. Создание через make_vector3d() ===
v3 (нулевой вектор) = (0.0, 0.0, 0.0)
Длина нулевого вектора: 0.0

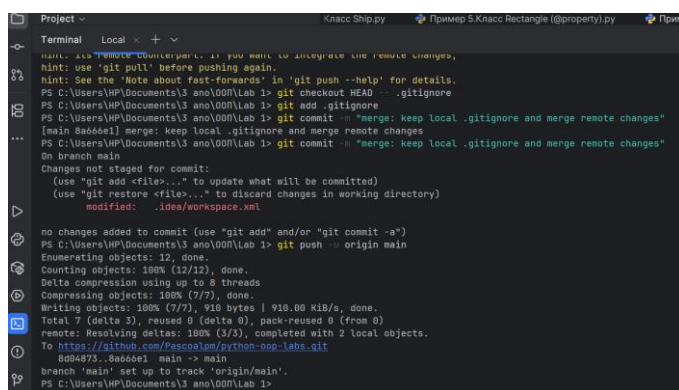
=== 8. Ввод с клавиатуры ===
Введите координаты x y z: 3 4 5
v4 (введённый) = (3.0, 4.0, 5.0)

=== 9. Проверка сравнения ===
|v_test| = 5.0 (должно быть 5.0)
v_test > zero: True
v_test == zero: False

```

Рисунок 8. Результат заданий 2

Организация репозитория с использованием модели ветвления git-flow и отправка изменений на GitHub.



```

Project v
Terminal Local x + v
Класс Ship.py
Пример 5.Класс Rectangle (@property).py
Прим

hint: to check out the remote changes,
hint: use 'git pull' before pushing again.
hint: See the 'Note about fast-forwards' in 'git push --help' for details.
PS C:\Users\HP\Documents\3 ano\OOP\Lab 1> git checkout HEAD -- .gitignore
PS C:\Users\HP\Documents\3 ano\OOP\Lab 1> git add .gitignore
PS C:\Users\HP\Documents\3 ano\OOP\Lab 1> git commit -m "merge: keep local .gitignore and merge remote changes"
(main 8a666e1) merge: keep local .gitignore and merge remote changes
PS C:\Users\HP\Documents\3 ano\OOP\Lab 1> git commit -m "merge: keep local .gitignore and merge remote changes"
On branch main
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
    modified:   .idea/workspace.xml

no changes added to commit (use "git add" and/or "git commit -a")
PS C:\Users\HP\Documents\3 ano\OOP\Lab 1> git push -u origin main
Enumerating objects: 12, done.
Counting objects: 100% (12/12), done.
Delta compression using up to 8 threads
Compressing objects: 100% (7/7), done.
Writing objects: 100% (7/7), 910 bytes | 910.00 KiB/s, done.
Total 7 (delta 3), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (3/3), completed with 2 local objects.
To https://github.com/Pascoalpin/python-op-labs.git
   8a64873..8a666e1  main -> main
branch 'main' set up to track 'origin/main'.
PS C:\Users\HP\Documents\3 ano\OOP\Lab 1>

```

Рисунок 9. Результат выполнения команды git push -u origin main

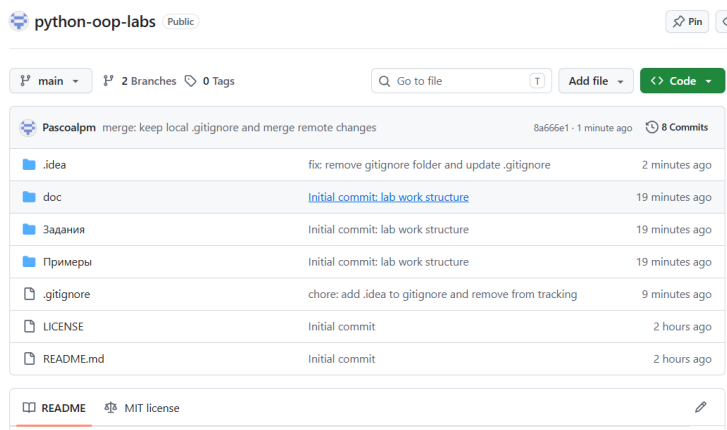


Рисунок 10. Структура репозитория на GitHub

Ответы на вопросы

1. Как осуществляется объявление класса в языке Python?

Класс в Python объявляется с помощью ключевого слова `class`, за которым следует имя класса и двоеточие. Тело класса содержит атрибуты и методы. Имена классов пишутся в стиле UpperCamelCase (с заглавной буквы, слова слитно без подчеркиваний).

Пример:

```
class MyClass:
    var = "значение"
    def do_smt(self):
        pass
```

2. Чем атрибуты класса отличаются от атрибутов экземпляра?

Атрибуты класса определяются внутри класса, но вне методов, и являются общими для всех экземпляров. Атрибуты экземпляра определяются в методах (обычно в `__init__`) с использованием `self` и являются уникальными для каждого объекта.

Пример:

```
class Pet:
    kind = "mammal"  # атрибут класса
    def __init__(self, name):
        self.name = name  # атрибут экземпляра
```

3. Каково назначение методов класса?

Методы класса определяют поведение объектов, принадлежащих этому классу. Они позволяют выполнять действия с данными объекта, изменять его состояние, взаимодействовать с другими объектами и реализовывать бизнес-логику приложения.

4. Для чего предназначен метод `init()` класса?

Метод `init()` является конструктором класса. Он предназначен для инициализации новых экземпляров, установки начальных значений атрибутов и настройки начального состояния объекта. Вызывается автоматически при создании объекта.

5. Каково назначение `self`?

`self` — это ссылка на конкретный экземпляр класса. Он представляет текущий объект, для которого вызывается метод, обеспечивает доступ к его атрибутам и методам, позволяет различать разные экземпляры класса. `self` всегда является первым параметром метода и передается неявно при вызове.

6. Как добавить атрибуты в класс?

Атрибуты можно добавить несколькими способами:

1. При создании класса: `class_attr = "значение"`
2. В методе `init`: `self.attr = значение`
3. В других методах: `self.new_attr = значение`
4. Напрямую к экземпляру: `obj.attr = значение`
5. После создания класса: `Class.new_attr = значение`

7. Как осуществляется управление доступом к методам и атрибутам в языке Python?

В Python нет строгого контроля доступа, используются соглашения:

- Публичный доступ: `attr` (доступен везде)
- Защищенный доступ: `_attr` (соглашение не использовать извне)

- Приватный доступ: `__attr` (name mangling — имя изменяется)

Для контроля доступа также используются свойства (`@property`) с геттерами и сеттерами.

8. Каково назначение функции `isinstance`?

Функция `isinstance(obj, cls)` проверяет, является ли объект `obj` экземпляром класса `cls` или его подкласса. Используется для проверки типа объекта во время выполнения, обеспечения корректности операций, обработки разных типов данных и реализации полиморфизма.

Пример:

```
isinstance(r1, Rational) # True или False
```

Вывод

В ходе выполнения лабораторной работы были освоены основные элементы объектно-ориентированного программирования в Python. Разработаны два класса согласно варианту 1:

1. Класс `Power` — для возведения числа в целочисленную степень с корректной обработкой всех краевых случаев.

2. Класс `Vector3D` — для работы с трехмерными векторами с реализацией всех арифметических операций и операций сравнения.

Все методы классов реализованы с контролем корректности входных данных и обработкой ошибок. Программы протестированы с использованием `pytest`, все тесты успешно пройдены.